

SOFT2412 NOTES FOR A HD

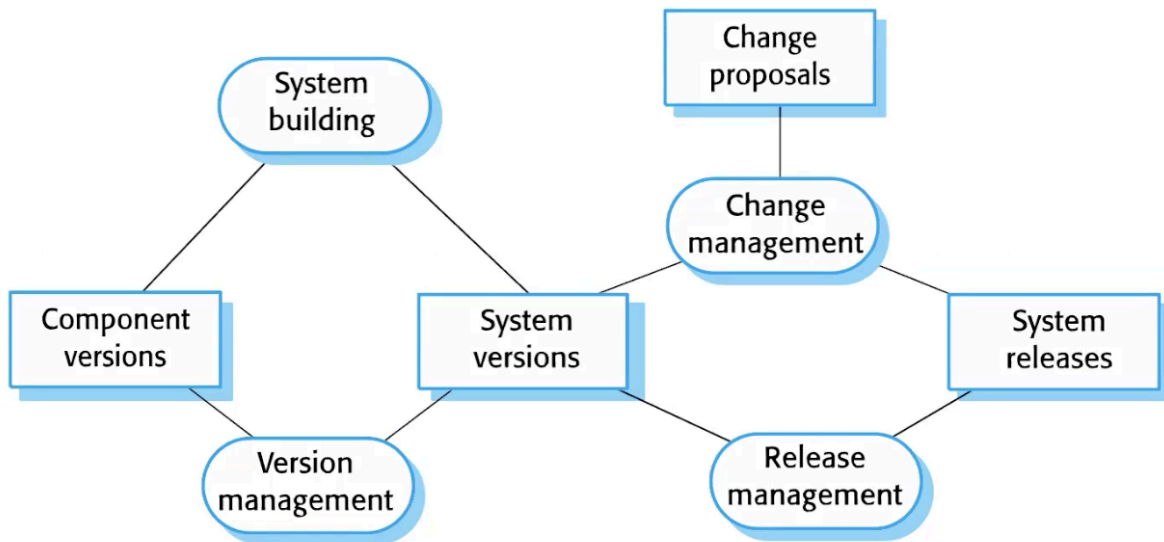
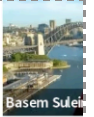
CONFIGURATION MANAGEMENT	1
VERSION MANAGEMENT	3
SEMANTIC VERSIONING (SemVer)	3
SYSTEM BUILDING	4
BUILD AUTOMATION: OTHER TOOLS	4
APACHE ANT	4
APACHE MAVEN	5
BUILD AUTOMATION: GRADLE	6
DEPENDS ON	7
TASK TYPES	7
EXEC	7
SUMMARISING “THE SCRUM GUIDE”	7
WHAT IS SCRUM?	7
SCRUM ARTEFACTS	9
PRODUCT BACKLOG	9
SPRINT BACKLOG	9
INCREMENT	9
DEFINITION OF DONE	10
SCRUM ROLES IN MORE DETAIL	10
PRODUCT OWNER	10
DEVELOPERS	10
SCRUM MASTER	11
SCRUM EVENTS	11
THE SPRINT	11
SPRINT PLANNING	11
DAILY SCRUM	12
SPRINT REVIEW	12
SPRINT RETROSPECTIVE	12
OTHER SCRUM THINGS	12
SCRUM ESTIMATION	12
PROGRESS MONITORING	12
USER STORIES	13
STORY POINTS	14
DEFINITION OF DONE	14
ETHICS	16
PROFESSIONAL FRAMEWORKS	16
ACS: CODE OF PROFESSIONAL CONDUCT	16
IEEE CS	16
SCENARIO	17
PRIVACY	18
CASE STUDY: TOYSMART (2000)	18
INTELLECTUAL PROPERTY (IP)	18
COPYRIGHT	18
TRADEMARKS	18

TRADE SECRETS	18
PATENTS	19
SCENARIO	19
OPEN SOURCE SOFTWARE	19

CONFIGURATION MANAGEMENT

- Concerned with the policies, processes and tools used for **managing changing software systems**.
- Track what changes and component versions are incorporated into each system version.
- Essential for team projects to control changes made by different developers.

Configuration Management Activities



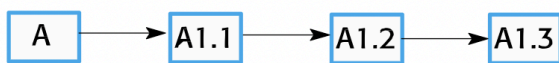
- **System building:** assembling program components, data and libraries, then compiling these to create an executable system.
- **Version management:** keeping track of the multiple versions of system components and ensuring that changes made to components by different developers do not interfere with each other
- **Change management:** keeping track of requests for changes to the software from customers and developers, working out the costs and impact of changes, and deciding if the changes should be implemented.
- **Release management:** preparing software for external release and keeping track of the system versions that have been released for customers.

VERSION MANAGEMENT

Version management is the process of managing *codelines* and *baselines* in order to keep track of different versions of software components and ensuring that changes made by different developers do not interfere with each other.

- **Codeline:** a sequence of versions of source code with later versions in the sequence derived from earlier versions
- **Baseline:** a definition of a specific system.
 - Specifies the component versions that are included in the system plus a specification of the libraries used, config files, etc.
 - Baselines may be specified using a configuration language, to define what components are included in a version of a particular system

Codeline (A)



Codeline (B)



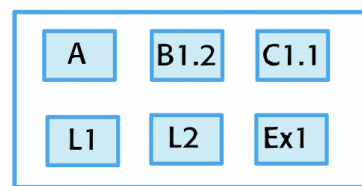
Codeline (C)



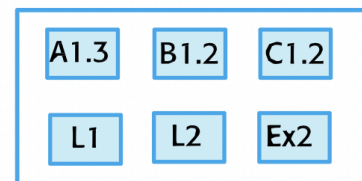
Libraries and external components



Baseline - V1



Baseline - V2



SEMANTIC VERSIONING (SemVer)

Given a version number **MAJOR**.**MINOR**.**PATCH**, increment the:

1. **MAJOR** version when you make incompatible API changes
2. **MINOR** version when you add functionality in a backwards-compatible manner
3. **PATCH** version when you make backwards-compatible bug fixes.

Stage	Code	Rule	Example
New product	1 st release	Start with 1.0.0	1.0.0
Patch Release	Bug fixes, other minor changes	Increment the 3 rd digit	1.0.1
Minor Release	New features that do not break existing features	Increment the 2 nd digit	1.1.0
Major Release	Changes that break backward compatibility	Increment the 1 st digit	2.0.0

For example, when coding in Python 2 you cannot run the program in Python 3, and you can't use anything after Python 3.7 with Python 3.6 code (needs to make some changes).

SYSTEM BUILDING

- System building is the process of creating a complete, executable system by compiling and linking the system components, external libraries, configuration files, etc.
- System building tools and version management tools must communicate as the build process involves checking out component versions from the repo managed by the version management system.
- The configuration description used to identify a baseline is also used by the system building tool.

BUILD AUTOMATION: OTHER TOOLS

Basically build automation is the act of scripting or automating a wide variety of tasks that software developers do in their day-to-day activities like:

1. Downloading dependencies.
2. Compiling source code into binary code.
3. Packaging that binary code.
4. Running tests.
5. Deployment to production systems.

APACHE ANT

Java-based software build tool for automating build processes

- Requires Java platform and best suited for building Java projects
- Does not impose coding conventions
- Does not impose any heavyweight dependency management framework
- XML to describe the code build process and its dependencies
 - Default *build.xml*

Drawbacks:

- Too flexible
- Complexity (XML-based build files)
- No standard structure/layout
 - Developers can create their own structure/layout of the project

APACHE MAVEN

- A build automation tool primarily for java projects
- XML-based description of the software being built
- Dependencies on other external modules and components, the build order, directories, and required plug-ins
- Central repository (e.g., Maven 2)
- Coding by convention: it uses conventions over configuration for the build procedure
- Supported by Eclipse, IntelliJ, JBuilder, NetBeans
- Plugin-based architecture

Drawbacks:

- Again, XML-based files increase complexity (verbose)
- Rigid; developers are required to understand follow the conventions

BUILD AUTOMATION: GRADLE

Gradle builds on the concepts of Apache Ant and Apache Maven, and introduces a Groovy- and Kotlin-based domain-specific language (DSL) contrasted with the XML-based project configuration used by Maven.

Gradle uses a directed acyclic graph to determine the order in which tasks can be run, through providing dependency management. It runs on the Java Virtual Machine.

- Task: a single atomic piece of work for a build – e.g., compiling classes
 - Each task has a name, which can be used to refer to the task within its owning project, and a fully qualified path, which is unique across all tasks in all projects
- Project: a composition of several tasks – e.g., Creation of a jar file.
 - Highest level Gradle concept
 - Contains one build script

Gradle has many optimisations built-in such as “incremental build.” This feature detects when the codebase hasn’t changed and so the next build is optimised to use less time.

Creating your own gradle tasks:

```
tasks.register("hello") {  
    println 'Hello World'  
}
```

Then if you run `gradle hello` on the terminal you will get the output “Hello World.” (wow)

DEPENDS ON

```
tasks.register("hello") {  
    dependsOn primaryTask  
    println 'Hello World'  
}  
  
tasks.register("primaryTask") {  
    println 'This is the primary task'  
}
```

Then running `gradle hello` will return

```
This is the primary task  
Hello World
```

Because the `hello` task **DEPENDS ON** the `primaryTask`, it must run all of `primaryTask` first.

TASK TYPES

There are many different types of tasks you can run in Gradle — these are kind of similar to functions...

EXEC

Executes a command line process.

<https://docs.gradle.org/current/dsl/org.gradle.api.tasks.Exec.html>

```
task stopTomcat(type:Exec) {
    workingDir '../tomcat/bin'

    //on windows:
    commandLine 'cmd', '/c', 'stop.bat'

    //on linux
    commandLine './stop.sh'

    //store the output instead of printing to the console:
    standardOutput = new ByteArrayOutputStream()

    //extension method stopTomcat.output() can be used to obtain the output:
    ext.output = {
        return standardOutput.toString()
    }
}
```

Each task has a bunch of “properties” such as “commandLine” and “standardOutput”.

SUMMARISING “THE SCRUM GUIDE”

WHAT IS SCRUM?

The Scrum Team consists of one Scrum Master, responsible for instilling and managing the Scrum methodology in the team, one Product Owner, responsible for the end product the team produces each Sprint, and Developers, those who are solely committed to creating a usable increment every Sprint.

There are essentially 4 steps to Scrum. The Scrum Master aims to create a collaborative environment where:

1. A Product Owner orders the work for a complex problem into a Product Backlog.
2. The Scrum Team turns a selection of the work into an Increment of value during a Sprint.
3. The Scrum Team and its stakeholders inspect the results and adjust for the next Sprint.
4. Repeat.

Some of the key tenets of Scrum are:

1. An iterative, incremental approach to producing a solution.
2. Transparency — all emerging work must be visible to those performing **and receiving** the work.
3. Inspection — Scrum artefacts and progress must be inspected frequently.
4. Adaptation — if any part of the Scrum, including the resulting product, is unacceptable, then the process being applied must be adjusted.
5. Empiricism — base what you will do next on what you know worked last time.

SCRUM ARTEFACTS

PRODUCT BACKLOG

The master to-do list of user stories, features, requirements, enhancements, and fixes which the Product Owner maintains. This list is ordered, and is the single source of truth for work to be undertaken by the Scrum Team.

PRODUCT GOAL:

The Product Goal is the long-term objective for the Scrum Team — the ideal future state of the product. The Product Goal sits inside the Product Backlog.

SPRINT BACKLOG

- The Sprint Backlog is composed of the Sprint Goal (why), the set of Product Backlog items selected for the Sprint (what), as well as an actionable plan for delivering the Increment (how).
- The Sprint Backlog is a plan by and for the Developers — it describes the work the Developers plan to accomplish during the Sprint in order to achieve the Sprint Goal.
- The Sprint Backlog is updated throughout the Sprint as more is learned.

INCREMENT

The usable end product from a Sprint.

- An Increment is a concrete stepping stone toward the Product Goal.
- In order to provide value, **the Increment must be usable**.
- Multiple Increments may be created within a Sprint, and an Increment may be delivered to stakeholders prior to the end of the Sprint.
- Work cannot be considered part of an Increment unless it meets the Definition of Done.

DEFINITION OF DONE

- The Definition of Done is a formal description of the state of the Increment when it meets the quality measures required for the product.
- The Definition of Done creates transparency by providing everyone a shared understanding of what work was completed as part of the Increment.
- If a Product Backlog item does not meet the Definition of Done, it cannot be released or even presented at the Sprint Review.
- The moment a Product Backlog item meets the Definition of Done, an Increment is initialised.

Essentially an “acceptance criteria” but for the product created at the end of the Sprint.

SCRUM ROLES IN MORE DETAIL

PRODUCT OWNER

The Product Owner is accountable for maximising the value of the product resulting from the work of the Scrum Team (The Scrum Guide, 2020).

- Has a very clear idea of the entire product and its end user requirements, and is responsible for communicating the Product Goal to the rest of the team.
- Have a vision for the product each sprint, placing them into user stories which are well communicated to the team.
- Responsible for maintaining Product Backlog and prioritising user stories.
- May represent the needs of many stakeholders.
- Communicate with Scrum Master regularly.

DEVELOPERS

Developers are the people in the Scrum Team that are committed to creating any aspect of a usable Increment each Sprint (The Scrum Guide, 2020).

Developers take part in:

- Creating a plan for the Sprint through the Sprint Backlog
- Ensuring the completion of a user story strictly meets a definition of 'done' (acceptance criteria).
- Working towards the Sprint Goal on a daily basis.

SCRUM MASTER

The Scrum Master is accountable for establishing Scrum in the Scrum Team.

Scrum masters are responsible for:

- Review and planning of the sprint as well as facilitating daily standups. The Scrum Master is also responsible for ensuring these "Scrum events" are positive, productive, and timely.
- Motivating the Team to produce high-value Increments that meet the definition of 'done'.
- Facilitating stakeholder collaboration as requested or needed.

Scrum masters are NOT responsible for:

- Managing developers on a day-to-day basis
- Managing or controlling the Sprint Backlog
- "Leading" the daily stand-ups

SCRUM EVENTS

Each event in Scrum is a formal opportunity to inspect and adapt Scrum artefacts.

THE SPRINT

This is the backbone of Scrum. All work necessary to achieve the Product Goal (planning, daily standups, review etc.) happen within Sprints.

- Fixed length, of one month or less. Usually the length of the Sprint depends on how frequently you meet the client.
- A new Sprint starts immediately after the end of the previous Sprint.
- The Product Backlog is refined as needed throughout the Sprint.
- Scope may be clarified and renegotiated with the Product Owner as more is learned.

SPRINT PLANNING

Sprint Planning initiates the Sprint by laying out the work to be performed for the Sprint. There are three main topics which must be discussed during each Planning:

1. Why is this Sprint valuable?
 - a. The Product Owner proposes how the product could increase its value and utility in the current Sprint.
 - b. The whole Scrum Team then collaborates to define a Sprint Goal that communicates why the Sprint is valuable to stakeholders.
 - c. The Sprint Goal must be finalised prior to the end of Sprint Planning.
2. What can be done this Sprint?
 - a. Through discussion with the Product Owner, the Developers select items from the Product Backlog to include in the current Sprint.
 - b. There can be many issues in this part such as deciding how much can be completed within a Sprint. The only way to get better at deciding this is through trial and error or “empiricism” which is one of Scrum’s central tenets.
3. How will the chosen work get done?
 - a. For each selected Product Backlog item, the Developers plan how to decompose this item into smaller work, usually of one day or less.
 - b. This allows Developers to plan “Increments” of work.

Generally Topics 1 and 2 are conducted in one meeting, and Topic 3 is undertaken in a separate meeting. This is known as the “two-parts meeting.”

The Sprint Goal, the Product Backlog items selected for the Sprint, plus the plan for delivering them are together referred to as the Sprint Backlog.

DAILY SCRUM

The purpose of the Daily Scrum is to inspect the current progress toward the Sprint Goal and adapt the Sprint Backlog as necessary, adjusting the upcoming planned work. It is usually 15 minutes or less, held at the same time every day of the Sprint.

SPRINT REVIEW

The Scrum Team presents the results of their work in the previous Sprint to key stakeholders/clients and progress toward the Product Goal is discussed.

The team presents and reviews what was accomplished in the Sprint to the client and key stakeholders. The overall progress towards the Product Goal is discussed, and a discussion is had about whether any requirements have changed since the last meeting. Based on this information, all attendees collaborate on figuring out what to do next. This may include adjusting the Product Backlog.

SPRINT RETROSPECTIVE

The Scrum Team inspects how the last Sprint went in regards to individuals, interactions, processes and tools. A discussion is held over what went right and what went wrong in the previous Sprint. Particularly important is to analyse the reasoning behind any problems encountered and to brainstorm any solutions or preventative measures to be added to the next Sprint.

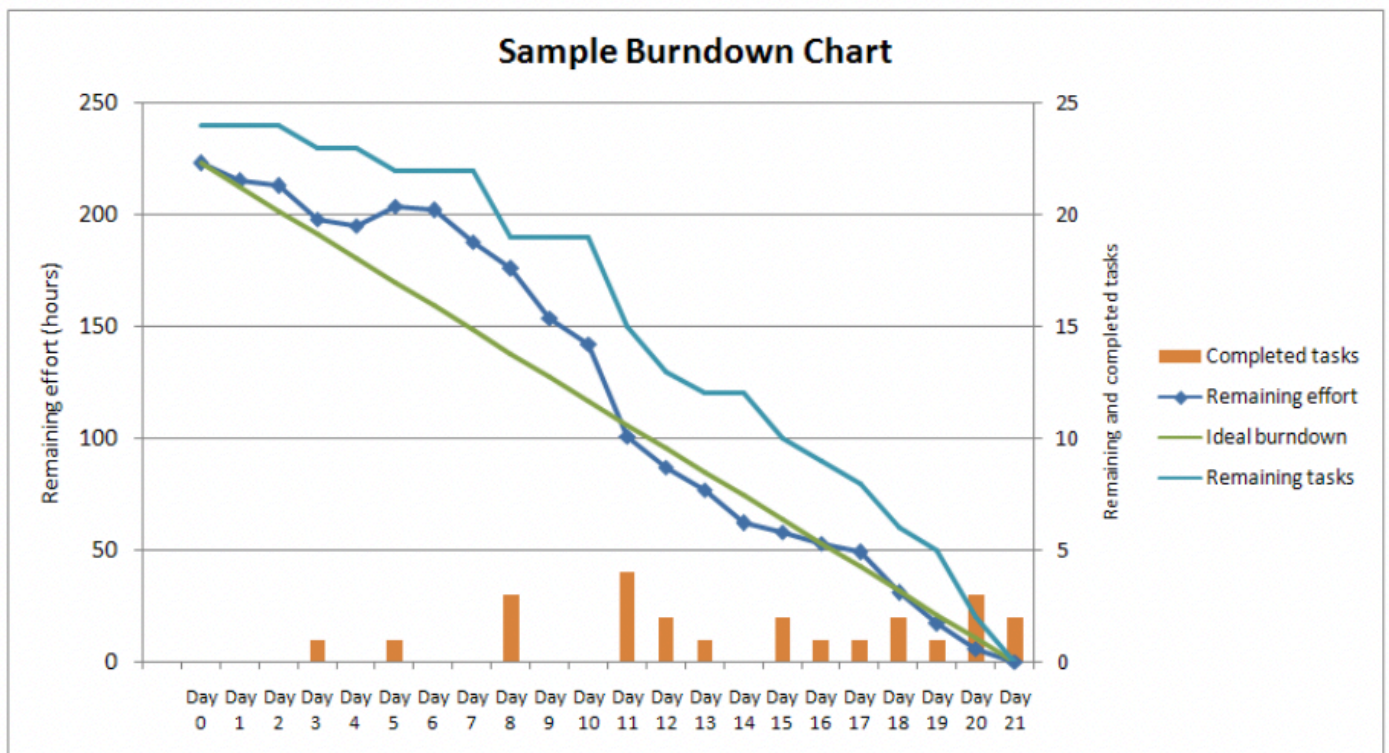
The Sprint Retrospective concludes the Sprint.

OTHER SCRUM THINGS

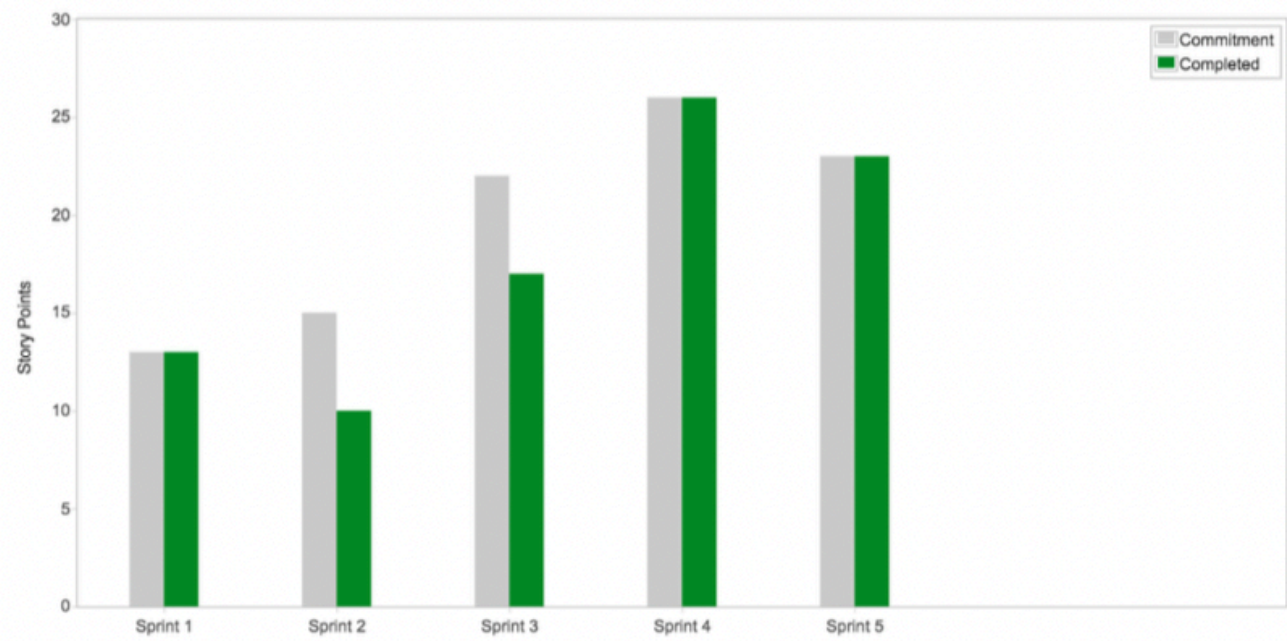
SCRUM ESTIMATION

PROGRESS MONITORING

- Total work remaining to reach the goal. The product owner tracks this at least every Sprint Review.
 - Compare with previous Sprints to assess progress towards projected work.
 - Forecasting progress through burn-downs, burn-ups or cumulative flows.
- “Burndown” chart tracks the amount of estimated effort remaining in the sprint.
 - Maintained on a daily basis by the Scrum Master
- “Velocity” charts tracks the amount of work done from Sprint to Sprint.
 - Sum the number of story points assigned to each user story that was completed during the sprint.



Velocity Chart



USER STORIES

As a [stakeholder], I want a [feature] so that [benefit].

E.g. As a movie fan, I want to add a movie to Rotten Potatoes, so that I can share movies with other movie fans.

E.g. As a returning user with a large friends list, I want to nominate one friend's video for an achievement, so that all of our mutual friends can vote to give him a star.

SMART User Stories:

- Specific
- Measureable
- Achievable
- Relevant
- Timeboxed

STORY POINTS

- Metric for the overall size of a user story, feature, or piece of work
- A point value is assigned to each item
- The smallest expected story is estimated at 1 story point, and the remaining user stories are numbered relative to this user story.

DEFINITION OF DONE

Acceptance criteria: an effective way for developers to gauge completion/satisfaction of features.

For example, for the user story:

As a returning user with a large friends list, I want to nominate one friend's video for an achievement, so that all of our mutual friends can vote to give him a star.

An acceptance criteria could be:

- A user can nominate a video for an achievement
- A user's friend is notified when his video gets an achievement
- A user can see all of the videos his friends have nominated
- A video with an achievement is displayed with a star next to it

ETHICS

PROFESSIONAL FRAMEWORKS

Most professional associations have “frameworks” guiding professional conduct.

For example:

- ACS: Code of Professional Conduct
- ACM Code of Ethics and Professional Conduct
- IEEE CS Code of Ethics

ACS: CODE OF PROFESSIONAL CONDUCT

- The Primacy of the Public Interest
 - You will place the interests of the public above those of personal, business or sectional interests.
- The Enhancement of Quality of Life
 - You will strive to enhance the quality of life of those affected by your work.
- Honesty
 - You will be honest in your representation of skills, knowledge, services and products.
- Competence
 - You will work competently and diligently for your stakeholders.
- Professional Development
 - You will enhance your own professional development, and that of your staff.
- Professionalism
 - You will enhance the integrity of the ACS and the respect of its members for each other.

IEEE CS

- **Public:** Software engineers shall act consistently with the public interest.
- **Client and employer:** Software engineers shall act in a manner that is in the best interests of their client and employer, consistent with the public interest.
- **Product:** Software engineers shall ensure that their products and related modifications meet the highest professional standards possible.
- **Judgement:** Software engineers shall maintain integrity and independence in their professional judgement.
- **Management:** Software engineering managers and leaders shall subscribe to and promote an ethical approach to the management of software development and maintenance.
- **Profession:** Software engineers shall advance the integrity and reputation of the profession consistent with the public interest.
- **Colleagues:** Software engineers shall be fair to and supportive of their colleagues.
- **Self:** Software engineers shall participate in lifelong learning regarding the practice of their profession and shall promote an ethical approach to the practice of the profession.

SCENARIO

You work for a small company, SmallCorp Consultants. The company’s only contract at present is a \$6M year-long project for Bluestone Mining to write a software system to analyse geological data.

You discover a way of designing the system that means that the project can be completed in 2 months, rather than 1 year, and at a cost of only \$1M.

Your boss tells you to keep quiet and ignore the “better solution” as they want to keep the team working on the project as long as possible (otherwise he would have to sack most of the staff), and Bluestone has already indicated they are happy to pay the \$6M as they believe the project is worth that much.

What do you do?

- Documenting the detailed investigation of the better solution with references to the professional framework and email it to your manager.
- Always refer to the code of conduct you are working with inside that company and the contract you signed when joining. Refer to this in your correspondence with your manager, but also know these policies so that you yourself aren't violating any rules in the way you deal with the situation.
- For example, your company's code of conduct may state you must not disclose any information to the client or reach out to the client directly to discuss any information with them.
- If your manager or the person you are reporting to is not taking any action, you may need to escalate the issue further to another department such as HR.
- Maintain professional conduct in your dealings with your manager and all parties involved. Remember that you are working for the client/the general interest of the public.

PRIVACY

ACM Professional Responsibility

- Access computing and communication resources only when authorised or when compelled by the public good
 - Individuals and organizations have the right to restrict access to their systems and data so long as the restrictions are consistent with other principles in the Code
 - Computing professionals should not access another's computer system, software, or data without a reasonable belief that such an action would be authorized or a compelling belief that it is consistent with the public good
- *“A system being publicly accessible is not sufficient grounds on its own to imply authorization. Under exceptional circumstances a computing professional may use unauthorized access to disrupt or inhibit the functioning of malicious systems; extraordinary precautions must be taken in these instances to avoid harm to others.”*

CASE STUDY: TOYSMART (2000)

- Online Toy store in Massachusetts, USA.
- Privacy policy on the website:
 - *Personal information, voluntarily submitted by visitors to our site, such as name, address, billing information and shopping preferences, is never shared with a third party.*
- When it ran into financial difficulties, however, it attempted to sell all of its assets, including its detailed customer databases.
- The FTC filed a lawsuit to prevent the sale of the customer information and alleged that Toysmart.com had misrepresented its privacy policy.
 - The FTC also alleged that Toysmart.com violated the Children's Online Privacy Protection Act of 1998 COPPA by collecting names, e-mail addresses, and ages of children under 13 without notifying parents or obtaining parental consent.
- Toysmart.com agreed to settle the case.

You need to respect the privacy policy that you offer your customers. A promise is still a promise, even in the IT world.

INTELLECTUAL PROPERTY (IP)

“A category of property that includes intangible creations of the human intellect.”

- IP is intangible unlike “traditional property”.
 - It’s therefore challenging to protect — how do you protect a software or design?
- IP law to protect people and business’ intellect

Below are the various types of IP.

COPYRIGHT

- Grant a creator of an original work exclusive right it
- Source code, executable code, database, artistic work
- Granted automatically in Australia, not all countries though
- Does not protect ideas or methods employed

TRADEMARKS

- Sign, design or expression which distinguishes products or services
- Formal registration and require meeting certain criteria

TRADE SECRETS

- Process, formula, practice, design, pattern which is unknown and provides a competitive advantage.

PATENTS

Form of IP protection which grants inventor(s) the right to exclude others from making, using or selling and importing an invention for a limited time, in exchange for the public disclosure of the invention

- Inventor(s):
 - Exclusive rights to the invention (usually 20 years)
 - Freedom to exploit the invention (commercially and non-commercially)
- Public:
 - Enrich body of knowledge and innovation – valuable information to be shared with the public
- To be patentable, an invention must be:
 - New
 - Non-obvious
 - Useful (industrial applicability)
 - The disclosure of the invention in the patent application must meet certain formal and substantive standards

SCENARIO

You accepted a 6-month short term contract job, working for a small start-up that is creating a new App that locates the cheapest place near your current location to buy consumer items. Whilst working on the project you accidentally discover that they have obtained access to proprietary code from another company (that analyses web pages to find product price information). They

aren't using the code directly, but they are analysing it so they can understand the algorithm and then duplicate it in their system.

The question is how did the company get access to the proprietary code? Is this legal?

OPEN SOURCE SOFTWARE

- Open-source: "something people can modify and share because its design is publicly accessible."
- Open-source software: "software with source code that anyone can inspect, modify, and enhance"

Does open-source software = free software?

- Free software refers to liberty of use, not price. Think "free speech" over "free beer."
- Run, copy, study, change, distribute and improve source code
- The four essentials of freedom:
 - Run the program as you wish for any purpose
 - Study how the program works, and change it as you wish
 - Redistribute copies of the original program
 - Redistribute copies of your modified versions